

Lecture 7 - Concurrency: Deadlocks

Emergency Crash Study Guide

Built for fast exam review: understand, memorize, and practice

Use this guide in this order if your exam is soon:

- 1) Read the one-page cheat sheet first.
- 2) Master the four deadlock conditions and the resource-allocation graph rules.
- 3) Learn the difference between prevention, avoidance, detection/recovery, and ignoring.
- 4) Practice the safe-state / Banker-style steps and the detection algorithm.
- 5) Finish with the trap questions at the end.

0. One-page exam cheat sheet

Concept	What you must say in the exam
Deadlock	A set of processes is blocked; each waits for an event that can only be caused by another blocked process in the same set.
Reusable resource	Used by one process at a time, not consumed. Examples: CPU, memory, files, databases, semaphores.
Consumable resource	Produced and consumed. Examples: messages, signals, interrupts, I/O buffer info. Blocking receive can deadlock.
Resource-allocation graph	Processes = circles, resources = squares, dots = instances, $P_i \rightarrow R_j$ means request, $R_j \rightarrow P_i$ means assignment.
Cycle rule	No cycle means no deadlock. Cycle means deadlock may exist; with single instance per resource, a cycle is deadlock. With multiple instances, a cycle may still be no deadlock.
Four conditions	Mutual exclusion, hold-and-wait, no preemption, circular wait. All four must hold. First three are necessary but not sufficient.
Prevention	Design system so deadlock is impossible by invalidating one of the four conditions. Conservative and can reduce resource utilization.
Avoidance	Allow conditions, but grant a request only if the resulting state is safe. Needs future max resource claims.
Safe state	There exists at least one order where every process can finish. Safe = no deadlock. Unsafe = possible deadlock, not necessarily deadlock.
Banker logic	Pretend to grant request $\rightarrow$ update state $\rightarrow$ check safe sequence $\rightarrow$ grant if safe, otherwise block.
Detection	Grant requests when possible, check later. If some processes cannot be marked as finishable, they are deadlocked.
Recovery	Abort deadlocked processes or preempt resources, with rollback/victim selection and starvation concerns.
Mixed strategy	Use different approaches for different resource classes: swap, files/tapes, memory, internal resources.

Memorize this sentence:

Deadlock needs all four: mutual exclusion + hold-and-wait + no preemption + circular wait. Prevention breaks a condition, avoidance stays safe, detection finds deadlock after it may happen, recovery breaks it.

1. What is deadlock?

Deadlock is not just waiting. Waiting becomes deadlock when every process in a set is blocked, and the event each process needs can only be caused by another process that is also blocked.

Normal waiting	Deadlock waiting
Process waits for disk I/O, timer, keyboard input, or a resource that will be released normally.	Process waits for something that cannot happen because the only process that could cause it is also waiting.
The system can still make progress.	The set of processes cannot make progress without external action.
Example: P1 waits for printer and P2 will finish printing soon.	Example: P1 holds disk and waits for tape; P2 holds tape and waits for disk.

Intersection example

The lecture uses cars at an intersection as an intuition. Potential deadlock happens when each car needs two quadrants. Actual deadlock happens when each car already occupies one quadrant and waits for the next quadrant, but that next quadrant is held by another car. Nobody can move.

Deadlock is usually about resources

Most deadlocks are caused by processes competing for a finite set of resources. The lecture emphasizes that there is no efficient general solution for every possible deadlock case, so operating systems use policies and algorithms depending on the resource type and system goals.

2. Resource types: reusable vs consumable

Resource type	Meaning	Examples	How deadlock happens
Reusable resources	Used by one process at a time and not depleted by use. A process obtains it, uses it, then releases it.	CPU, memory, files, databases, semaphores, tape drive, disk file.	Each process holds one reusable resource and requests another held by someone else.
Consumable resources	Created/produced and destroyed/consumed by processes.	Messages, signals, interrupts, information in I/O buffers.	A process blocks waiting to receive a message that will never be sent because the sender is also blocked.

Exam trap:

Reusable-resource deadlock is often about holding resource A while waiting for B.

Consumable-resource deadlock is often a design error, like both processes doing blocking receive before either sends.

Reusable resource example

P and Q compete for disk file D and tape drive T. P locks D and then requests T. Q locks T and then requests D. If the schedule is P0, P1, Q0, Q1, P2, Q2, both processes are stuck: P waits for T, Q waits for D.

Memory deadlock example

Available memory is 200 KB. P1 gets 80 KB and later requests 60 KB. P2 gets 70 KB and later requests 85 KB. After the first requests, only 50 KB remains. Neither second request can be satisfied. This can be handled using virtual memory.

Consumable resource example

```
P1: Receive(P2);  Send(P2, Message1);  
P2: Receive(P1);  Send(P1, Message2);
```

If Receive is blocking, both processes block before either sends. That is a deadlock caused by bad message-passing design.

3. Resource-allocation graphs

A resource-allocation graph is a directed graph used to describe resource ownership and waiting. It is one of the most exam-friendly parts of this lecture.

Symbol / edge	Meaning
Circle: P_i	A process.
Square: R_j	A resource type.
Dot inside square	One instance of that resource type. Three dots means three instances.
$P_i \rightarrow R_j$	Request edge: process P_i is requesting resource R_j and is waiting.
$R_j \rightarrow P_i$	Assignment edge: an instance of R_j is currently held by P_i .
Cycle	A closed chain of waiting/holding relationships. It signals possible deadlock.

Cycle rules

Graph condition	Conclusion
No cycle	No deadlock.
Cycle and each resource type has only one instance	Deadlock exists.
Cycle but some resource types have multiple instances	Deadlock may exist, but it is not guaranteed. Another instance may allow progress.
Cycle disappears after some process can finish and release resources	No deadlock.

Professor trap:

Do not automatically say every cycle means deadlock. The safe answer is: no cycle $\rightarrow$ no deadlock; cycle $\rightarrow$ deadlock may exist. If each resource has a single instance, then a cycle implies deadlock.

How to analyze a graph quickly

1. List all request edges: $P_i \rightarrow R_j$.
2. List all assignment edges: $R_j \rightarrow P_i$.
3. Trace arrows to see if you return to the same process/resource. That is a cycle.
4. Check the number of resource instances. If there are multiple dots, a cycle may still be safe.
5. Explain who is waiting for whom using plain English.

Example with deadlock

A graph may have cycles such as $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$ and $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$. In that case, P_1 , P_2 , and P_3 are deadlocked because each is waiting for a resource held by another process in the cycle.

Example with cycle but no deadlock

The lecture also shows a graph with a cycle $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$ but no deadlock. The reason is that the resource types have multiple instances, and other processes can finish and release resources, breaking the cycle.

4. The four necessary conditions for deadlock

Condition	Meaning	Easy example
1. Mutual exclusion	Only one process may use a resource at a time.	Only one process can use a printer or write to a file at a time.

2. Hold-and-wait	A process holds at least one resource while waiting for another.	P1 holds disk D and waits for tape T.
3. No preemption	The OS cannot forcibly take the resource away. The process must release it voluntarily.	You cannot safely steal a printer halfway through a print job.
4. Circular wait	A closed chain exists: each process holds a resource needed by the next process.	P1 waits for P2, P2 waits for P3, P3 waits for P1.

Must-know wording:

The first three conditions are necessary but not sufficient. All four conditions must hold for deadlock. Irresolvable circular wait is basically the definition of deadlock.

How exam questions usually test the four conditions

- They describe a scenario and ask which condition is missing. If one condition is missing, no deadlock can occur.
- They ask why a solution prevents deadlock. Answer by saying which condition it breaks.
- They ask why a solution is inefficient. Explain resource underutilization, delays, preemption cost, or starvation.

5. Addressing deadlock: the big map

Approach	Main idea	When it acts	Tradeoff
Prevention	Make deadlock impossible by breaking one necessary condition.	Before deadlock can happen.	Conservative; may waste resources or delay processes.
Avoidance	Grant a request only if the resulting state is safe.	At each allocation request.	Needs future max claims; more concurrency than prevention.
Detection and recovery	Allow deadlocks, periodically detect them, then recover.	After deadlock may happen.	More liberal; recovery can be costly.
Ignore	Pretend deadlocks do not occur.	No automatic action.	Used by many OSs including UNIX; manual restart may be needed.

Fast comparison:

Prevention says: Do not let the conditions exist.

Avoidance says: Conditions may exist, but never enter an unsafe state.

Detection says: Let it happen, then find and fix it.

Ignore says: It is cheaper to do nothing automatically.

6. Deadlock prevention

Deadlock prevention invalidates at least one of the four necessary conditions. There are two styles: indirect prevention breaks mutual exclusion, hold-and-wait, or no preemption; direct prevention prevents circular wait.

6.1 Prevent mutual exclusion

To prevent deadlock by attacking mutual exclusion, make resources shareable when possible. This is not always possible. For example, read-only files can be shared, but writable files, semaphores, printers, and other non-sharable resources still require mutual exclusion.

Idea	Problem
Do not allow exclusive ownership.	Not applicable to many resources. Some resources are inherently non-sharable.
Share read-only resources.	Does not solve write/update resources.

6.2 Prevent hold-and-wait

Hold-and-wait means a process is holding resources while requesting more. To prevent it, force a process not to hold anything while requesting resources.

Solution	How it works	Disadvantages
Request all resources before starting	A process starts only after all resources it will need are granted.	May not know needs in advance; process delayed; resources tied up while idle.
Release all before requesting new ones	Before requesting another resource, release everything held, then request all needed resources again.	Low resource utilization; starvation possible.

6.3 Prevent no preemption

To prevent deadlock by attacking no preemption, allow resources to be forcibly taken back. This only works for resources whose state can be saved and restored easily, such as the processor. It is bad for resources like printers or file updates.

Preemption method	Why it is costly
If a process is denied a new request, force it to release its current resources and request again later.	The process may lose progress; inefficient.
If P1 requests a resource held by P2, preempt P2 and give the resource to P1.	May waste work and can be unsafe for non-preemptable resources.

6.4 Prevent circular wait

Circular wait is prevented by imposing a total order on all resource types. Processes must request resources in increasing order.

Assign every resource type a number: $R_1 < R_2 < R_3 < \dots$

A process holding R_i may request R_x only if $R_i < R_x$ for every resource R_i it holds.

If it needs a lower-numbered resource, it must release higher/lower conflicting resources first.

This prevents cycles because all requests move in one direction through the ordering. The cost is that it may slow processes and deny resource access unnecessarily.

Most testable prevention answer:

If the question asks how to prevent circular wait, answer: impose a total ordering of resource types and force processes to request resources in increasing order.

7. Deadlock avoidance

Deadlock avoidance is less restrictive than prevention. It allows the first three necessary conditions, but before granting a request, the OS checks whether the request would leave the system in a safe state.

Key idea

- If granting the request keeps the system safe -> grant it.
- If granting the request makes the system unsafe -> block the process, even if the requested resources are currently available.
- Avoidance requires advance information about each process maximum resource needs.

State variables in the lecture

Symbol	Meaning
$R = (R_1, R_2, \dots, R_m)$	Total amount of each resource in the system.
$V = (V_1, V_2, \dots, V_m)$	Available vector: amount of each resource not allocated to any process.
C matrix	Claim matrix: C_{ij} is the maximum requirement/claim of process i for resource j .
A matrix	Allocation matrix: A_{ij} is the current allocation to process i of resource j .
$C - A$	Need matrix: how much more each process may still need to complete.

Relationships that must hold

$R_j = V_j + \sum_i (A_{ij})$ for every resource j
 $C_{ij} \leq R_j$ no process can claim more than total resources
 $A_{ij} \leq C_{ij}$ allocated amount cannot exceed the claimed maximum

Safe state vs unsafe state

State	Meaning	Important trap
Safe state	There exists at least one sequence where all processes can finish.	Safe state means no deadlock.
Unsafe state	No guaranteed sequence exists using current assumptions.	Unsafe does not mean deadlock now. It means deadlock is possible.
Deadlocked state	Processes are actually blocked in a circular wait.	Every deadlocked state is unsafe, but not every unsafe state is deadlocked.

Critical distinction:

Unsafe state = danger zone. It is not automatically deadlock.

Avoidance does not predict deadlock with certainty. It prevents entering states where deadlock could become possible.

Two avoidance approaches

Approach	How it works	Weakness
Process initial denial	Start a new process only if the maximum claims of all current processes plus the new process can be met.	Assumes worst case: everyone asks for maximum at once. Not optimal.

Resource allocation denial	Do not grant an incremental request if the new state would be unsafe. This is the Banker-style idea.	Needs declared maximum claims and fixed resources.
----------------------------	--	--

8. Banker-style safety test

The Banker algorithm checks whether a state is safe. It is called banker because a banker should not lend money in a way that prevents all customers from eventually finishing.

Safety test steps

6. Compute $\text{Need} = \text{Claim} - \text{Allocation}$.
7. Set $\text{Work} = \text{Available}$.
8. Find a process not finished such that $\text{Need}_i \leq \text{Work}$.
9. Pretend it finishes. Add its Allocation_i back to Work .
10. Repeat until either all processes finish or no process can be found.
11. If all processes can finish, the state is safe. If not, it is unsafe.

Resource request decision

12. Check if $\text{Request}_i + \text{Allocation}_i \leq \text{Claim}_i$. If not, it is an error because the process asked for more than its declared maximum.
13. Check if $\text{Request}_i \leq \text{Available}$. If not, the process must wait.
14. Pretend to grant the request: $\text{Allocation}_i += \text{Request}_i$ and $\text{Available} -= \text{Request}_i$.
15. Run the safety test.
16. If safe, actually grant. If unsafe, restore old state and block the process.

Pseudocode from the lecture, rewritten cleanly

```

if Allocation[i] + Request[i] > Claim[i]:
    error # process asked for more than its maximum claim
else if Request[i] > Available:
    suspend process
else:
    pretend to allocate resources
    if safe(new_state):
        carry out allocation
    else:
        restore old state
        suspend process

```

Worked safe-state example from the lecture

Four processes P1-P4 and three resources R1-R3 are given. Claim matrix C, Allocation matrix A, $\text{Need} = \text{C} - \text{A}$, Resource vector R, and Available vector V are:

Process	C: R1 R2 R3	A: R1 R2 R3	Need C-A: R1 R2 R3
P1	3 2 2	1 0 0	2 2 2
P2	6 1 3	6 1 2	0 0 1
P3	3 1 4	2 1 1	1 0 3
P4	4 2 2	0 0 2	4 2 0

Vector	R1	R2	R3
Total resources R	9	3	6
Available V	0	1	1

Safe sequence

Step	Available before	Process that can finish	Why	Available after release
1	0 1 1	P2	Need P2 = 0 0 1 $\leq$ 0 1 1	6 2 3
2	6 2 3	P1	Need P1 = 2 2 2 $\leq$ 6 2 3	7 2 3
3	7 2 3	P3	Need P3 = 1 0 3 $\leq$ 7 2 3	9 3 4
4	9 3 4	P4	Need P4 = 4 2 0 $\leq$ 9 3 4	9 3 6

Therefore, the state is safe because at least one complete finishing order exists: P2, P1, P3, P4.

When should we grant a request?

The lecture asks whether P1 should be granted 1 unit of R1 and 1 unit of R3. The answer is no because granting it can lead to an unsafe state. The system may not be deadlocked immediately, but deadlock becomes possible because the remaining processes may each need at least one more R1 and none may be available.

Restrictions of avoidance

- Maximum resource requirements must be stated in advance.
- Processes must be independent: no ordering or synchronization requirements between them.
- The number of resources must be fixed.
- No process may exit while holding resources.

9. Deadlock detection

Detection is the opposite philosophy from prevention. Instead of restricting resource requests strongly, the system grants resource requests whenever possible and periodically checks whether a deadlock exists.

When to run detection

Option	Advantage	Disadvantage
At every request	Detects deadlock early; algorithm can be based on small changes.	High CPU overhead.
Periodically	Lower overhead.	Deadlock may remain longer before detection.
When CPU utilization drops below a threshold	Useful signal that many processes may be blocked.	May detect late.

Detection data structures

Symbol	Meaning
A	Allocation matrix. A_{ij} = resources of type j currently allocated to process i.
V	Available vector. Resources currently free.
Q	Request matrix. Q_{ij} = resources of type j currently requested by process i.
W	Temporary available vector used by the algorithm. Initially $W = V$.
Marked process	A process that is proven not deadlocked because it can finish.
Unmarked process at end	Deadlocked process.

Detection algorithm

17. Mark every process whose allocation row is all zeros. Such a process holds no resources, so it cannot be part of a deadlock.
18. Set temporary vector $W = \text{Available } V$.
19. Find an unmarked process i such that $Q_i \leq W$. This means its outstanding request can be satisfied.
20. If found, mark process i and pretend it finishes by adding its allocation back: $W = W + A_i$. Then repeat step 3.
21. If no such process exists, stop.
22. If any processes remain unmarked, they are deadlocked.

Detection rule:

If all processes can eventually be marked -> no deadlock.

If some processes remain unmarked -> those processes are deadlocked.

Avoidance vs detection trap:

Avoidance uses Claim/Need and refuses unsafe requests before deadlock.

Detection uses current Request Q and Allocation A to find deadlock after it may happen.

Detection example logic from the lecture

In one example, P4 is marked first because it has no allocated resources. Then P3 can be marked because its request is $\leq$ available. After releasing P3 resources, no other process can be marked. Therefore P1 and P2 are deadlocked.

10. Recovery from deadlock

Detection only tells you that a deadlock exists. Recovery is how the OS breaks it.

10.1 Process termination

Method	Meaning	Cost / issue
Abort all deadlocked processes	Kill every process involved in the deadlock.	Simple but expensive; some processes may have nearly finished.
Abort one process at a time	Kill one victim, rerun detection, repeat until deadlock disappears.	Requires repeated detection and careful victim selection.

Victim selection factors

- Process priority.
- How long the process has already computed and how much remains.
- Resources the process has used.
- Resources the process still needs to complete.
- How many processes will need to be terminated.
- Whether the process is interactive or batch.

Important warning:

Aborting a process is not always clean. If it was updating a file, the file may be left in an inconsistent state.

10.2 Resource preemption

The OS may preempt resources from one or more deadlocked processes and give them to others until the cycle is broken.

Issue	Explanation
Selecting a victim	Choose which process/resource to preempt while minimizing

	cost. Criteria include resources held and time consumed.
Total rollback	Abort the process and restart it. Simple but inefficient.
Partial rollback	Return process to a safe checkpoint and restart from there. Needs the OS to store more process state.
Starvation	The same process might always be chosen as victim. Prevent this by limiting how many times a process can be picked or including rollback count in cost.

11. Mixed strategy

A real OS can use different deadlock strategies for different resource classes. This is often better than using one policy for everything.

Resource class	Possible strategy
Swap space	Prevention by requiring all swap space to be allocated at once; avoidance can also be possible.
Tapes/files	Avoidance can work because processes may declare needs ahead of time; prevention by resource ordering is also possible.
Main memory	Prevention by preemption is a good approach.
Internal resources such as channels	Prevention by ordering resources.
Between classes	A linear ordering between classes can be used.

12. Advantages and disadvantages summary

Approach	Resource allocation policy	Major advantages	Major disadvantages
Prevention: request all resources at once	Conservative; undercommits resources.	Works well for processes that perform one burst of activity; no preemption needed.	Inefficient; delays process initiation; future resource needs must be known.
Prevention: preemption	Conservative.	Convenient when resource state can be saved and restored easily.	Preempts more often than necessary.
Prevention: resource ordering	Conservative.	Feasible via compile-time checks; no run-time computation if solved in system design.	Disallows incremental resource requests.
Avoidance	Midway between detection and prevention.	No preemption necessary; more concurrency than prevention.	Future resource needs must be known; processes can be blocked for long periods.
Detection	Very liberal; requested resources are granted where possible.	Never delays process initiation; supports online handling.	Recovery causes preemption/rollback losses.

13. Exam traps and how to answer them

Trap question	Correct answer
Does a cycle always mean deadlock?	No, not when resource types can have multiple instances. A cycle means deadlock may exist. With one instance per resource type, a cycle implies deadlock.
Does unsafe state mean deadlock?	No. Unsafe means deadlock is possible. The system may still be able to avoid it depending on future requests/releases.
Are the first three conditions enough for deadlock?	No. They are necessary but not sufficient. Circular wait must also hold.
Why is prevention conservative?	It restricts resource requests even when deadlock might not have happened. This can reduce utilization and delay processes.
Why does avoidance need future information?	It must know maximum claims to decide whether a request leaves a safe state.
Why is detection more liberal?	It grants resources whenever possible and checks later, instead of blocking requests that may only be dangerous.
Why can recovery be expensive?	It may abort or roll back work, preempt resources, leave files inconsistent, and cause starvation.
Why can blocking receive cause deadlock?	If both processes wait to receive before sending, no message is ever produced.

14. Practice questions with answers

Q: Define deadlock.

A: A set of processes is deadlocked when every process in the set is blocked waiting for an event that can only be triggered by another blocked process in the same set.

Q: List the four necessary conditions.

A: Mutual exclusion, hold-and-wait, no preemption, circular wait.

Q: Why are the first three conditions not sufficient?

A: Because without circular wait, the waiting chain can still be broken and some process may eventually progress.

Q: What is a reusable resource?

A: A resource used by one process at a time and not depleted by use, such as memory, files, databases, semaphores, CPU, or tape drives.

Q: What is a consumable resource?

A: A resource created and destroyed by processes, such as messages, signals, interrupts, or buffer information.

Q: What does $P_i \rightarrow R_j$ mean in a resource-allocation graph?

A: Process P_i requests resource R_j .

Q: What does $R_j \rightarrow P_i$ mean?

A: An instance of resource R_j is assigned to process P_i .

Q: What is a safe state?

A: A state where there is at least one sequence in which all processes can finish without deadlock.

Q: What is an unsafe state?

A: A state where no guaranteed safe finishing sequence exists. It is a possibility of deadlock, not necessarily a current deadlock.

Q: What is the main idea of Banker algorithm?

A: Pretend to grant a request, then check if the resulting state is safe. Grant only if safe.

Q: What is process initial denial?

A: A process is started only if its maximum claim plus the claims of current processes can be met.

Q: What is resource allocation denial?

A: An incremental request is denied or blocked if granting it might lead to unsafe/deadlock state.

Q: What data does detection use?

A: Allocation matrix A, available vector V, request matrix Q, and temporary work vector W.

Q: How do you know which processes are deadlocked in detection?

A: At the end, any unmarked processes are deadlocked.

Q: Name two recovery approaches.

A: Terminate processes or preempt resources.

Q: Why can victim selection cause starvation?

A: If the same process is repeatedly chosen for rollback or preemption, it may never finish.

15. Last-minute memorization page

Deadlock conditions mnemonic: M-H-N-C

M = Mutual exclusion

H = Hold and wait

N = No preemption

C = Circular wait

Approach mnemonic: P-A-D-I

P = Prevention: break a condition.

A = Avoidance: never enter unsafe state.

D = Detection/recovery: find and fix after it happens.

I = Ignore: cheaper to pretend it does not happen in many systems.

If you see...	Think...
P holds A, asks for B; Q holds B, asks for A	Classic reusable-resource deadlock.
Receive before send on both sides	Consumable-resource/message deadlock.
Graph has no cycle	No deadlock.
Graph has cycle + one instance per resource	Deadlock.
Graph has cycle + multiple instances	Possible deadlock; analyze more.
Need $\leq$ Available	That process can finish in Banker/detection reasoning.
Some processes cannot be marked in detection	Those unmarked processes are deadlocked.
System blocks even though resource is available	Avoidance may be refusing an unsafe allocation.

Final confidence checklist:

- I can define deadlock in one sentence.
- I can explain the intersection/car example.
- I can distinguish reusable and consumable resources.

- I can draw/interpret request and assignment edges.
- I know the cycle rules for resource-allocation graphs.
- I can list and explain the four conditions.
- I can compare prevention, avoidance, detection/recovery, and ignoring.
- I can run a safe-state check using $\text{Need} = \text{Claim} - \text{Allocation}$.
- I can run the detection marking algorithm.
- I can explain termination, preemption, rollback, victim selection, and starvation.